AULA 06 / 27/11/2026

O caminho dos dados

GET, POST, HTTPS e contrato do projeto integrador

Prof. Fabrício Rodrigues - Curso Técnico em Informática

Objetivos

• Distinguir uma simulação local de uma requisição a um serviço.

• Comparar GET e POST e explicar o papel do HTTPS.

• Documentar campos e regras de um serviço futuro sem inventar um backend.

Ponto de partida

Retome o projeto e a explicação da aula anterior. Se não tiver os arquivos, baixe a base deste encontro: ela contém o ponto de partida necessário.

Mapa da aula

Situação	Representação
Busca de dados com GET	Parâmetros podem aparecer na URL.
Envio com POST	Dados de formulário podem seguir no corpo.
HTTPS	Protege o tráfego em trânsito.
Projeto no GitHub Pages	Site estático; não implementa recebimento de chamados.

Os limites do protótipo

Até aqui o JavaScript trabalha somente na página. Mostrar um resumo não faz uma requisição de envio. O GitHub Pages publica os arquivos estáticos do curso; não executa um serviço próprio para receber chamados.

Uma aplicação real precisaria de uma rota de servidor e de regras de recebimento. Nesta aula documentaremos esse contrato e compararemos representações, sem enviar dados a um endereço externo.

GET e os parâmetros da URL

GET costuma ser usado para recuperar informações. Em um formulário com method=get, os pares de dados podem compor a query da URL. Eles podem aparecer no histórico e ser compartilhados junto com o endereço.

O exemplo gera uma string com URLSearchParams e a mostra como texto. exemplo.test é um domínio de exemplo; o projeto não navega para ele. Observe como nomes, espaços e caracteres são representados.

```
const dados = new FormData(form);
const parametros = new URLSearchParams(dados);
const exemplo = 'https://exemplo.test/chamados?'
  + parametros.toString();
// Apenas exibir o texto. Não navegar para esse endereço.
```

POST, HTTPS e conferência no servidor

Em um formulário HTML com method=post, os dados podem seguir no corpo da requisição. POST sozinho não criptografa dados. HTTPS protege o tráfego em trânsito. Não coloque informações sensíveis em uma URL por achar que o método resolveu toda a proteção.

O servidor deve conferir formato, tamanho, escolhas permitidas e demais regras. O usuário pode alterar a página ou enviar outra requisição. A validação HTML e o JavaScript servem à experiência local e precisam de conferência no serviço real.

Contrato do integrador

Documente no HTML um bloco de contrato: finalidade, método escolhido, rota proposta, campos e regras a aplicar no servidor. Identifique claramente que a rota não está implementada.

A base exibe exemplos de URL GET e de um corpo para POST depois da simulação. São representações de estudo. Complete o contrato e confronte seus nomes de campos com o que FormData realmente coleta.

```
<!-- Exemplo de contrato para uma aplicação futura.
Método: POST. Rota proposta: /chamados.
O servidor deve validar os campos novamente.
Este projeto estático não implementa essa rota.
-->
```

Prática no VS Code

Baixe o projeto específico desta aula. Extraia o ZIP, abra a pasta no VS Code e execute index.html com Live Server. Os três arquivos são index.html, style.css e script.js.

1. Abrir a base da aula e observar o resumo e as representações de estudo.

2. Comparar onde os valores apareceriam em GET e POST e explicar por que não houve envio real.

3. Completar o contrato no HTML: finalidade, POST, rota proposta /chamados, campos, regras e indicação de serviço ainda não implementado.

4. Conferir se os names coletados correspondem ao contrato e registrar o escopo do integrador.

Casos de teste

Compare o esperado com o observado. Registre falhas e correções no TXT da aula.

Caso	Resultado esperado
Simulação com dados fictícios	Resumo e representações locais, sem envio.
Espaços no nome	URL de exemplo com parâmetros codificados.
Dois sintomas selecionados	Representação de POST preserva a lista.
Contrato sem indicar implementação futura	Revisar para não prometer serviço existente.

Caso	Resultado esperado
POST sem HTTPS no argumento	Explicar que POST não oferece criptografia.
Campo sem name	Identificar a ausência na coleta e corrigir o contrato.

Erros que merecem atenção

• O corpo de POST não é uma garantia de criptografia.

• Não afirmar que o Pages recebeu, validou ou armazenou o chamado.

• Um contrato é uma proposta; não é prova de que o endpoint existe.

Conferência

[ ] Implementei os requisitos da atividade desta aula.

[ ] Testei os casos da tabela e registrei o resultado observado.

[ ] Consigo explicar uma decisão e uma correção.

[ ] Guardei o ZIP e o TXT com nome, turma e aula.

Uma explicação para fechar

Qual parte do seu projeto roda no navegador? O que ainda faltaria num serviço real?

Entrega

Pasta com index.html, style.css e script.js em ZIP, acompanhada do registro de testes em TXT.

Da aula 02 em diante: salve os três arquivos em ZIP, abra o Registro de testes no portal, informe nome e turma e baixe o TXT. Entregue ZIP e TXT ao professor na aula. Não há envio nem salvamento automático.

Referências com links

GitHub: funcionamento do Pages

MDN: envio de dados (em inglês)

MDN: HTTPS (em inglês)

MDN: uso de FormData (em inglês)

VS Code: arquivos e pastas (em inglês)

Live Server de Ritwick Dey: abrir o projeto (em inglês)